

Laboratorium 9: Kalibracja prawdopodobieństw

Zadanie 1: Kalibracja prawdopodobieństw dla „toy set”

Cel: Implementacja kodu do kalibracji wyjść z sieci w oparciu o metodę „temperature scaling”

Studenci implementują:

1. Przygotowanie zbioru „toy set” oraz podział na części train_val, kalibracyjną i testową

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Data
X, y = make_classification(n_samples=5000, n_features=20, random_state=42)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

X_train_val_cal, X_test, y_train_val_cal, y_test = train_test_split(X, y, test_size=0.2)
```

2. Trening sieci do klasyfikacji w schemacie walidacji krzyżowej, zapisanie najlepszych modeli na zbiorach walidacyjnych

3. Implementacja i trening modułu do skalowania temperatury – kalibrację wykonujemy niezależnie dla wszystkich pięciu modeli z ensembła z pkt. 2:

```
class TemperatureScaler(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model
        model.eval()
        self.temperature = nn.Parameter(torch.ones(1)) # T

    def forward(self, x):
        with torch.no_grad():
            logits = self.model(x)
        return logits / self.temperature

def fit_temperature(model, X_val, y_val):
    scaler = TemperatureScaler(model)
    optimizer = optim.LBFGS([scaler.temperature], lr=0.001, max_iter=15000)
    criterion = nn.CrossEntropyLoss()

    def closure():
        optimizer.zero_grad()
        logits = scaler(X_val)
        loss = criterion(logits, y_val)
        loss.backward()
        return loss

    optimizer.step(closure)
    return scaler
```

```
scaler = fit_temperature(model, X_cal, y_cal)
```

4. Wyznaczenie i wizualizacja krzywych kalibracyjnych przed i po kalibracji:

```
# Before calibration
logits = model(X_val).detach()
probs_before = F.softmax(logits, dim=1)[:, 1].numpy()

# After temperature scaling
logits_scaled = scaler(X_val).detach()
probs_after = F.softmax(logits_scaled, dim=1)[:, 1].numpy()

# --- Compute calibration curves ---
frac_pos_before, mean_pred_before = calibration_curve(
    y_true, probs_before, n_bins=10
)

frac_pos_after, mean_pred_after = calibration_curve(
    y_true, probs_after, n_bins=10
)
```

Na wykresie krzywej kalibracyjnej na osi poziomej odkłada się średnie prawdopodobieństwa, a na osi pionowej odkłada się frakcje przypadków pozytywnych.

5. Wyznaczenie wartości Brier score przed i po kalibracji:

```
from sklearn.metrics import log_loss, brier_score_loss
brier_score_loss(y_true, probs_before)
```

Zadanie: Wykonaj kalibrację klasyfikatora w oparciu o zbiór „toy set”. Zaprezentuj wyniki w postaci krzywych kalibracyjnych i wartości „Brier score”

Zadanie 2: Ocena niepewności predykcji przed i po kalibracji

Cel: Ocena wpływu kalibracji na niepewność predykcji

- # 1. Studenci oceniają dokładność wytrenowanych ensembli przed i po wykonaniu kalibracji. Wyniki należy porównać wykonując test McNemara
- # 2. Studenci wyznaczają rozkłady całkowitej niepewności predykcji (entropia) przed i po normalizacji.
- # 3. Studenci wyznaczają rozkłady niepewności epistemicznej przed i po normalizacji.
- # 4. Studenci wyznaczają rozkłady niepewności aleatorycznej przed i po normalizacji.

Zadanie: Porównaj dokładność oraz rozkład niepewności klasyfikacji przed i po wykonaniu kalibracji prawdopodobieństw.

Zadanie 3: Wrapper scikit-learn do modelu pytorch

Cel: Przygotowanie wrappera scikit-learn do modelu pytorch w celu wykorzystania metod kalibracji udostępnianych przez scikit-learn

Studenci implementują kod wrappera do modułu pytorch. Konstruktor wrappera powinien przyjmować przynajmniej poniższą listę argumentów:

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.utils.validation import check_is_fitted

class MyWrapper(ClassifierMixin, BaseEstimator):

    def __init__(
        self,
        model,
        classes,
        lr=1e-3,
        epochs=50,
        batch_size=None,
        device=None,
        is_fitted = False
    ):
```

Wrapper powinien móc przyjmować zarówno wytrenowany jak i niewytrenowany model pytorch:

```
# model jest wytrenowanym klasyfikatorem
clf = MyWrapper(model=model, classes = np.unique(y_train), is_fitted = True)
```

Wrapper powinien implementować metody fit, predict, predict_proba, score. Sposoby użycia:

```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import cross_val_score

clf.fit(X_train, y_train)
y_pred = clf.predict(X_val)
scores = cross_val_score(clf, X_val, y_val, cv=5)

print(confusion_matrix(y_val, y_pred))
print(accuracy_score(y_val, y_pred))
print(scores)
```

Sposób wykorzystania wrappera do kalibracji wytrenowanego modelu:

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator

#calibrated_clf = CalibratedClassifierCV(FrozenEstimator(clf),method='sigmoid')
#calibrated_clf = CalibratedClassifierCV(FrozenEstimator(clf),method='isotonic')
calibrated_clf = CalibratedClassifierCV(FrozenEstimator(clf),method='temperature')
# skalowanie metodą ' temperature' jest dostępne od wersji scikit-learn 1.8.0

init_probs = clf.predict_proba(X_val)
```

```

calibrated_probs = calibrated_clf.predict_proba(X_val)
y_true = y_val.numpy()

# --- Compute calibration curves ---
frac_pos_before, mean_pred_before = calibration_curve(
    y_true, init_probs[:,1], n_bins=10
)

frac_pos_after, mean_pred_after = calibration_curve(
    y_true, calibrated_probs[:,1], n_bins=10
)

```

Zadanie: Wykonaj kalibrację wytrenowanego modelu pytorch z wykorzystaniem infrastruktury scikit-learn. Porównaj jakość kalibracji (np. używając Brier score) z wykorzystaniem trzech metod udostępnianych przez scikit-learn

Zadanie 4: Kalibracja klasyfikatora dla rzeczywistego zbioru danych

Cel: Wykonanie kalibracji prawdopodobieństw klasyfikatora dla rzeczywistego zbioru danych

Studenci wykorzystują metody z zadań 1-3 do wytrenowania i wykalibrowania klasyfikatora dla rzeczywistego zbioru danych (np. pobranego z openml)

Zadanie: Wytrenuj i wykalibruj klasyfikator dla rzeczywistego zbioru danych. Oceń dokładność predykcji oraz rozkład niepewności przed i po kalibracji.